

AgriTech Farm Management System

Project Documentation

Full Stack Development with MERN

1. Introduction

Project Title	AgriTech Farm Management System
Technology Stack	MongoDB, Express.js, React.js, Node.js (MERN)
Version	1.0.0
Project Type	Full Stack Web Application (Capstone Project)

1.1 Team Members and Roles

The following team members contributed to the development of this project:

Role	Responsibilities
Full Stack Developer	System architecture, backend API development, database design
Frontend Developer	React component development, UI/UX implementation
Backend Developer	REST API endpoints, authentication, middleware
Database Administrator	MongoDB schema design, data modeling
Project Lead	Project management, documentation, testing coordination

2. Project Overview

2.1 Purpose

The AgriTech Farm Management System is a comprehensive web-based platform designed to empower Filipino farmers with digital tools for modern agricultural management. The system bridges the gap between traditional farming practices and modern technology by providing an integrated suite of tools accessible through any web browser.

The platform addresses the following core needs of the agricultural community:

- Centralized management of farm operations and crop tracking
- Real-time weather data integration for informed farming decisions
- A community forum for knowledge sharing among farmers
- An online marketplace for buying and selling agricultural products
- Analytics and reporting tools for yield optimization
- Role-based access for farmers, experts, and administrators

2.2 Key Features

- **Farm Management - Register and manage multiple farms with location and soil type data**
 - Add, edit, and delete farm records
 - View farm details and associated crops
- **Crop Management - Track crop lifecycle from planting to harvest**
 - Log planting dates, expected harvest dates, and growth status
 - Monitor crop health and growth stages
- **Weather Dashboard - Real-time weather data via OpenWeatherMap API**
 - Current temperature, humidity, wind speed
 - Weather-based farming recommendations
- **Community Forum - Discussion board for farmers and agricultural experts**
 - Create posts, comment, and share farming knowledge
- **Marketplace - Buy and sell agricultural products**
 - Product listings with images and booking system
 - Order tracking and payment confirmation
- **Analytics - Data visualization for farm performance**
 - Yield charts and weather trend graphs
- **Admin Dashboard - Complete system administration panel**
 - Manage users, products, bookings, and forum posts
- **User Authentication - Secure JWT-based login and registration**

3. Architecture

3.1 Frontend Architecture (React.js)

The frontend is built with React 18 using Vite as the build tool. The application uses React Router v6 for client-side routing and implements a Context API-based authentication system.

Framework	React 18.2.0 with Vite 4.4.5
Routing	React Router DOM v6
State Management	React Context API (AuthContext)
UI Library	React Bootstrap 5.3 + React Icons
Charts	Recharts 2.x for analytics visualization
HTTP Client	Axios for API communication
Notifications	React Toastify for user feedback
Date Handling	Moment.js

The frontend follows a component-based architecture organized by feature domains. Protected routes use a PrivateRoute wrapper that verifies authentication and redirects unauthenticated users to the login page.

3.2 Backend Architecture (Node.js + Express.js)

The backend is a RESTful API server built with Express.js 4.18 running on Node.js 18+. It follows the MVC (Model-View-Controller) pattern with separate layers for routes, controllers, models, and middleware.

Runtime	Node.js 18 or higher
Framework	Express.js 4.18.2
Authentication	JWT (jsonwebtoken 9.0.2) + bcryptjs
File Uploads	Multer (max 5MB, images only)
Environment	dotenv for configuration management
HTTP Client	Axios (for weather API calls)
Dev Tools	Nodemon for development hot reload

3.3 Database (MongoDB)

The application uses MongoDB Atlas as the cloud database provider. Mongoose 7.5 is used as the ODM (Object Data Modeling) library for schema definition and data validation.

Core data models include:

- User - Stores farmer, admin, and expert accounts with hashed passwords
- Farm - Farm records linked to a user with location and land area data
- Crop - Crop records linked to farms with growth tracking fields
- Product - Marketplace product listings with images and pricing
- Booking - Order records linking users to products with status tracking
- ForumPost / ForumComment - Community discussion data
- ExpertAdvice - Expert consultation records

4. Setup Instructions

4.1 Prerequisites

Ensure the following software and accounts are available before installation:

Node.js	Version 18 or higher (https://nodejs.org)
MongoDB Atlas	Free cloud account (https://mongodb.com/atlas)
OpenWeatherMap API	Free API key (https://openweathermap.org/api)
npm	Comes bundled with Node.js
Web Browser	Any modern browser (Chrome, Firefox, Edge)

4.2 Environment Variables

Configure the backend environment by editing the `.env` file located in the `backend/` folder. Set the following variables:

```
MONGODB_URI=mongodb+srv://USERNAME:PASSWORD@cluster.mongodb.net/agritech?retryWrites=true&w=majority
JWT_SECRET=any_random_secret_string_you_choose
PORT=5000
OPENWEATHER_API_KEY=your_openweathermap_api_key_here
```

4.3 Installation Steps

Step 1: Install Backend Dependencies

```
cd MERN_CAPSTONE_FINAL/backend
npm install
```

Step 2: Install Frontend Dependencies

```
cd MERN_CAPSTONE_FINAL/frontend
npm install
```

Step 3: Create Admin Account (Run Once)

```
cd MERN_CAPSTONE_FINAL/backend
node create-admin.js
```

Default admin credentials: `admin@agritech.com` / `admin123`

5. Folder Structure

5.1 Client (React Frontend)

```
frontend/
  index.html          <- App HTML entry point
  vite.config.js      <- Vite build configuration
  package.json        <- Frontend dependencies
  src/
    App.jsx           <- Root component with routing
    main.jsx          <- React DOM entry point
    index.css         <- Global styles
    pages/            <- Full page components
      Home.jsx        <- Landing page
      Login.jsx       <- Login page
      Register.jsx    <- Registration page
      FarmerDashboard.jsx <- Farmer home dashboard
      AdminDashboard.jsx <- Admin control panel
      FarmsPage.jsx   <- Farm management
      CropsPage.jsx   <- Crop tracking
      MarketplacePage.jsx <- Product marketplace
      ForumPage.jsx   <- Community forum
      AnalyticsPage.jsx <- Analytics & charts
      MyOrdersPage.jsx <- Order tracking
    components/       <- Reusable UI components
      Common/         <- Shared: PrivateRoute, ErrorMessage, Spinner
      Dashboard/      <- Weather widget, alerts, status cards
      Layout/         <- Navbar, Sidebar, Footer
      Profile/        <- ProfilePage, Settings
    context/
      AuthContext.jsx <- Global authentication state
    services/         <- Axios API service functions
```

5.2 Server (Node.js Backend)

```
backend/
  server.js           <- Express app entry point
  create-admin.js     <- Script to seed admin user
  package.json        <- Backend dependencies
  .env               <- Environment variables (not committed)
  config/
    db.js            <- MongoDB connection setup
  controllers/        <- Route handler logic
    authController.js <- Login & registration
    adminController.js <- Admin CRUD operations
    cropController.js <- Crop management logic
    farmController.js <- Farm management logic
    productController.js <- Marketplace logic
```

```
forumController.js <- Forum post/comment logic
weatherController.js <- OpenWeatherMap integration
expertController.js <- Expert advice logic
middleware/
  auth.js           <- JWT verification middleware
  roleCheck.js      <- Role-based access control
models/            <- Mongoose data schemas
  User.js, Farm.js, Crop.js, Product.js
  Booking.js, ForumPost.js, ForumComment.js
routes/            <- Express route definitions
uploads/           <- Uploaded image storage
```


6. Running the Application

6.1 Start the Backend Server

Open a terminal in the backend directory and run:

```
cd MERN_CAPSTONE_FINAL/backend  
npm run dev
```

Expected output: Server running on port 5000 | MongoDB Connected

The backend API will be available at: <http://localhost:5000>

6.2 Start the Frontend

Open a second terminal in the frontend directory and run:

```
cd MERN_CAPSTONE_FINAL/frontend  
npm run dev
```

Expected output: VITE ready | Local: <http://localhost:5173>

6.3 Access the Application

Frontend URL	http://localhost:5173
Backend API URL	http://localhost:5000/api
Health Check	http://localhost:5000/api/health
Admin Login	admin@agritech.com / admin123
Farmer Account	Register at /register

7. API Documentation

All API endpoints are prefixed with /api. Protected endpoints require a Bearer token in the Authorization header.

7.1 Authentication Endpoints (/api/auth)

Endpoint	Method	Description	Auth Required	Parameters
/api/auth/register	POST	Register a new farmer account	No	name, email, password
/api/auth/login	POST	Login and receive JWT token	No	email, password
/api/auth/me	GET	Get current logged-in user profile	Yes (Bearer)	None
/api/auth/profile	PUT	Update user profile	Yes (Bearer)	name, phone, bio, location

7.2 Farm Endpoints (/api/farms)

Endpoint	Method	Description	Auth Required	Parameters
/api/farms	GET	Get all farms for current user	Yes	None
/api/farms	POST	Create a new farm	Yes	name, location, area, soilType
/api/farms/:id	GET	Get a specific farm by ID	Yes	Farm ID (URL param)
/api/farms/:id	PUT	Update farm details	Yes	Farm fields to update
/api/farms/:id	DELETE	Delete a farm record	Yes	Farm ID (URL param)

7.3 Crop Endpoints (/api/crops)

Endpoint	Method	Description	Auth Required	Parameters
/api/crops	GET	Get all crops for current user	Yes	None
/api/crops	POST	Add a new crop record	Yes	name, farmId, plantedDate, status
/api/crops/:id	GET	Get a specific crop	Yes	Crop ID (URL param)
/api/crops/:id	PUT	Update crop information	Yes	Crop fields to update
/api/crops/:id	DELETE	Delete a crop record	Yes	Crop ID (URL param)

7.4 Marketplace / Product Endpoints (/api/products)

Endpoint	Method	Description	Auth Required	Parameters
/api/products	GET	Get all available products	Yes	None
/api/products	POST	Create a product listing	Yes	name, price, description, images
/api/products/:id	GET	Get product details	Yes	Product ID
/api/products/:id	PUT	Update product listing	Yes	Product fields
/api/products/:id	DELETE	Remove a product	Yes	Product ID

7.5 Booking Endpoints (/api/bookings)

Endpoint	Method	Description	Auth Required	Parameters
/api/bookings	POST	Create a new booking/order	Yes	productId, quantity, deliveryDate
/api/bookings	GET	Get all bookings for current user	Yes	None
/api/bookings/:id	GET	Get a specific booking	Yes	Booking ID
/api/bookings/:id/cancel	PUT	Cancel a booking	Yes	Booking ID
/api/bookings/:id/confirm-payment	PUT	Confirm payment for booking	Yes	Booking ID

7.6 Forum Endpoints (/api/forum)

Endpoint	Method	Description	Auth Required	Parameters
/api/forum	GET	Get all forum posts	Yes	None
/api/forum	POST	Create a new forum post	Yes	title, content, category
/api/forum/:id	GET	Get post with comments	Yes	Post ID
/api/forum/:id/comments	POST	Add comment to a post	Yes	content
/api/forum/:id	DELETE	Delete a forum post	Yes	Post ID

7.7 Weather Endpoints (/api/weather)

Endpoint	Method	Description	Auth Required	Parameters
/api/weather	GET	Get weather by city name	Yes	city (query param)

Endpoint	Method	Description	Auth Required	Parameters
/api/weather/coords	GET	Get weather by coordinates	Yes	lat, lon (query params)

7.8 Admin Endpoints (/api/admin)

Endpoint	Method	Description	Auth Required	Parameters
/api/admin/users	GET	Get all users in system	Yes (Admin)	None
/api/admin/users/:id	DELETE	Delete a user account	Yes (Admin)	User ID
/api/admin/products	GET	Get all marketplace products	Yes (Admin)	None
/api/admin/bookings	GET	Get all bookings system-wide	Yes (Admin)	None
/api/admin/forum	GET	Get all forum posts	Yes (Admin)	None

7.9 File Upload Endpoints

Endpoint	Method	Description	Auth Required	Parameters
/api/upload/single	POST	Upload a single image file	Yes	image (multipart/form-data)
/api/upload/multiple	POST	Upload up to 10 images	Yes	images (multipart/form-data)
/api/health	GET	Server health check	No	None

8. Authentication

8.1 Authentication Method

The application uses JSON Web Tokens (JWT) for stateless authentication. Upon successful login or registration, the server generates a signed JWT token that the client stores in `localStorage`.

8.2 Authentication Flow

1. User submits login credentials (email + password) to `POST /api/auth/login`
2. Server verifies the email and compares the bcrypt-hashed password
3. On success, server returns a signed JWT with the user ID embedded
4. Frontend stores the token in `localStorage` and user data in `AuthContext`
5. All subsequent requests include: `Authorization: Bearer <token>`
6. The auth middleware verifies the token and attaches the user to `req.user`

8.3 Role-Based Authorization

Role	Access Level
farmer (default)	Access to own farms, crops, marketplace, forum, and analytics
admin	Full system access including AdminDashboard and all management endpoints
expert	Access to expert advice endpoints and farmer consultations

8.4 Password Security

Passwords are never stored in plain text. The system uses `bcryptjs` with a salt factor of 10 to hash all passwords before storing them in MongoDB. Password comparison is done securely using `bcrypt.compare()`.

9. User Interface

9.1 Application Pages

Page / Route	Description
/ (Home)	Public landing page introducing the AgriTech platform with navigation to login/register
/login	Login form with email and password fields; redirects based on user role on success
/register	Farmer registration form with name, email, and password
/dashboard	Farmer dashboard with weather widget, crop status cards, and quick stats
/farms	Farm management page with add/edit/delete functionality
/crops	Crop tracking page with growth status and planting/harvest dates
/marketplace	Product listing page with booking modal for purchasing products
/my-orders	Order history and status tracking for the logged-in farmer
/forum	Community forum with post list, post details, and comment section
/analytics	Analytics dashboard with yield charts and weather trend graphs
/profile	User profile page with editable personal information and profile picture
/admin	Full admin panel for managing users, products, bookings, and forum posts

9.2 UI Technology Highlights

- Responsive design using React Bootstrap grid and components
- Navigation via a top Navbar with role-based menu items
- Real-time toast notifications for all user actions (React Toastify)
- Loading spinners for asynchronous data fetching states
- Interactive charts rendered with Recharts for analytics
- Image preview for product listings and profile pictures

10. Testing

10.1 Testing Strategy

The project was tested using a combination of manual functional testing and API testing throughout the development cycle.

10.2 API Testing

- All REST endpoints were tested using a browser-based API client (e.g., Thunder Client or Postman)
- Authentication flows were verified by testing token generation, expiry, and invalid token rejection
- CRUD operations for all modules (farms, crops, products, bookings, forum) were tested with valid and invalid inputs
- Admin-only routes were tested with both admin and farmer tokens to verify role enforcement

10.3 Frontend Testing

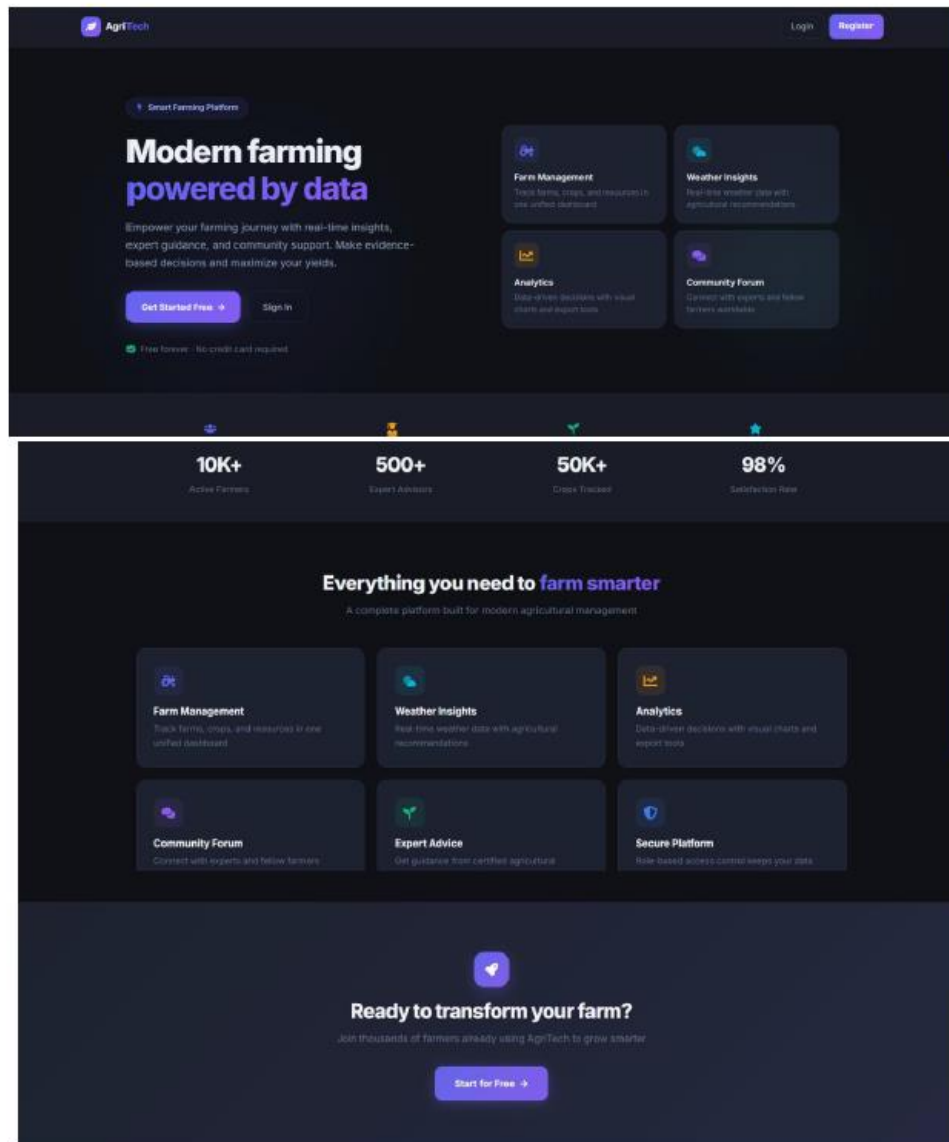
- Manual testing of all page routes including protected route redirection behavior
- Form validation tested with missing fields, invalid emails, and short passwords
- Booking flow tested end-to-end: browse product -> book -> view in My Orders -> cancel
- Weather widget tested with valid city names and error handling for invalid locations

10.4 Tools Used

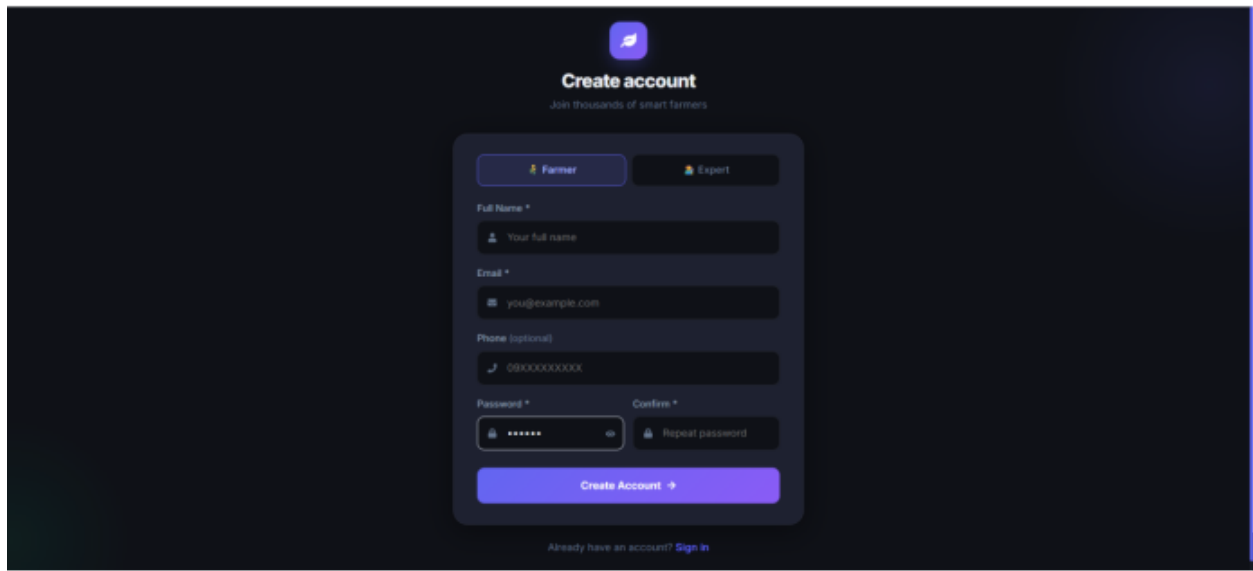
API Testing	Thunder Client / Postman (manual endpoint testing)
Browser Testing	Google Chrome DevTools (network inspection, console errors)
Database Validation	MongoDB Atlas UI (data verification)

11. Screenshots or Demo

Landing Page:

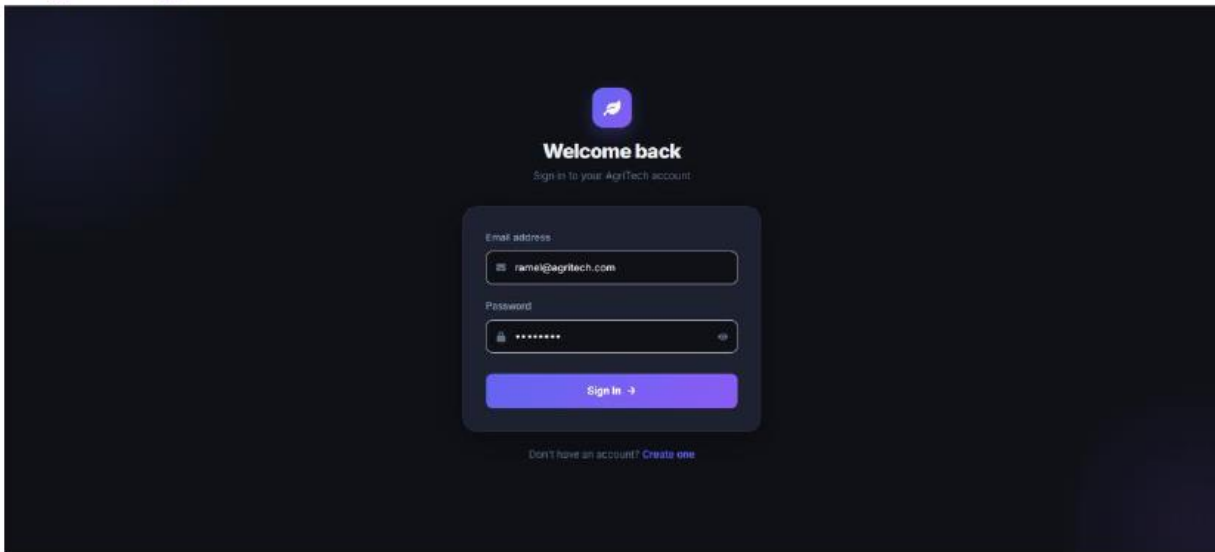


Registration Page:

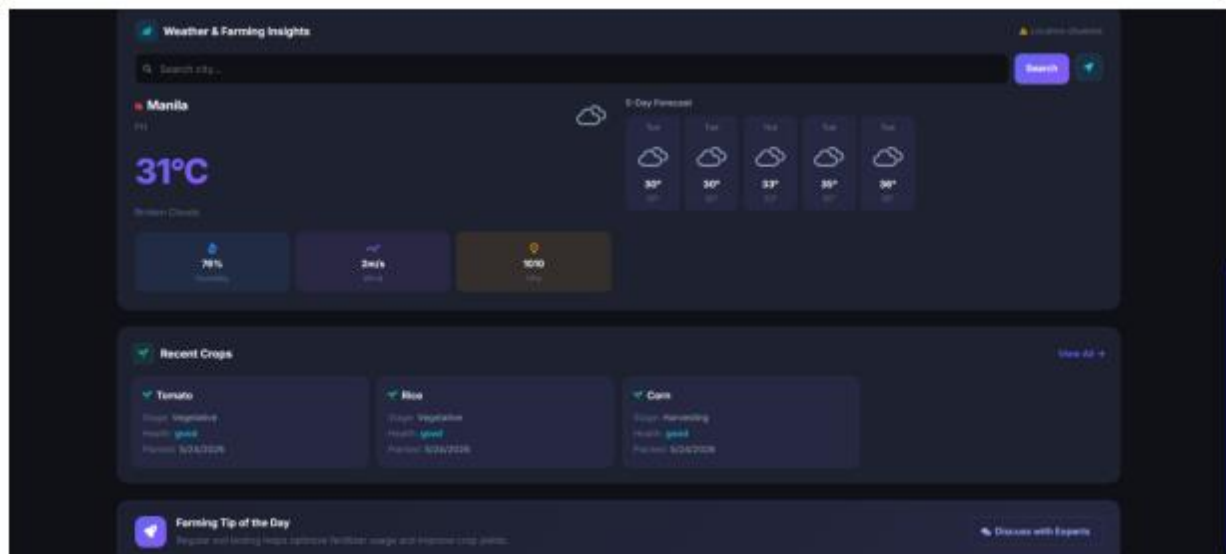
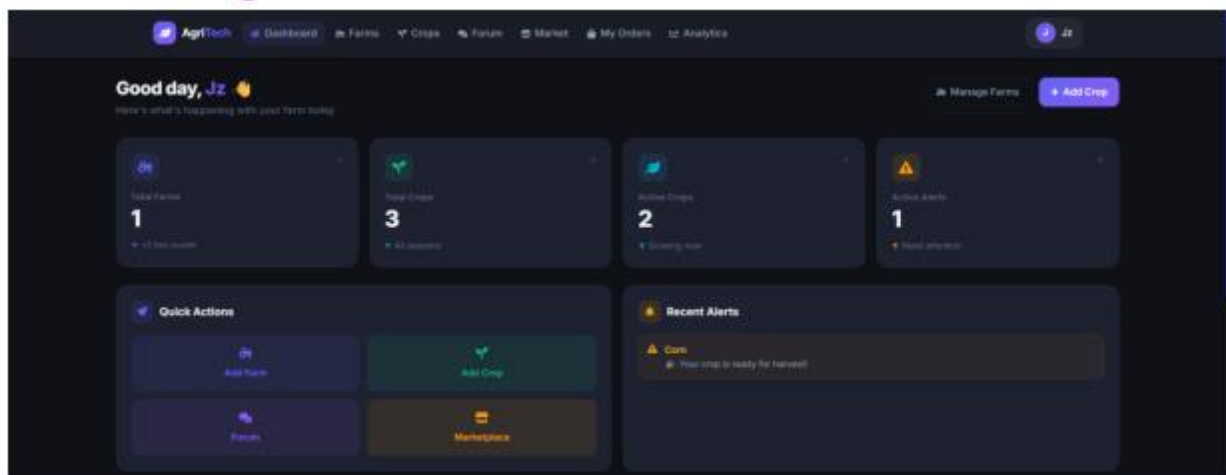


The image shows a registration page with a dark blue background. At the top center is a purple square icon with a white leaf. Below it, the text "Create account" is displayed in white, followed by the subtitle "Join thousands of smart farmers" in a smaller font. The registration form is a light blue rounded rectangle containing two tabs: "Farmer" (selected) and "Expert". The form fields include: "Full Name *" with a placeholder "Your full name"; "Email *" with a placeholder "you@example.com"; "Phone (optional)" with a placeholder "09000000000"; "Password *" with a masked input "*****" and a toggle icon; and "Confirm *" with a placeholder "Repeat password". A large blue "Create Account" button with a right arrow is at the bottom of the form. Below the button, the text "Already have an account? [Sign in](#)" is displayed.

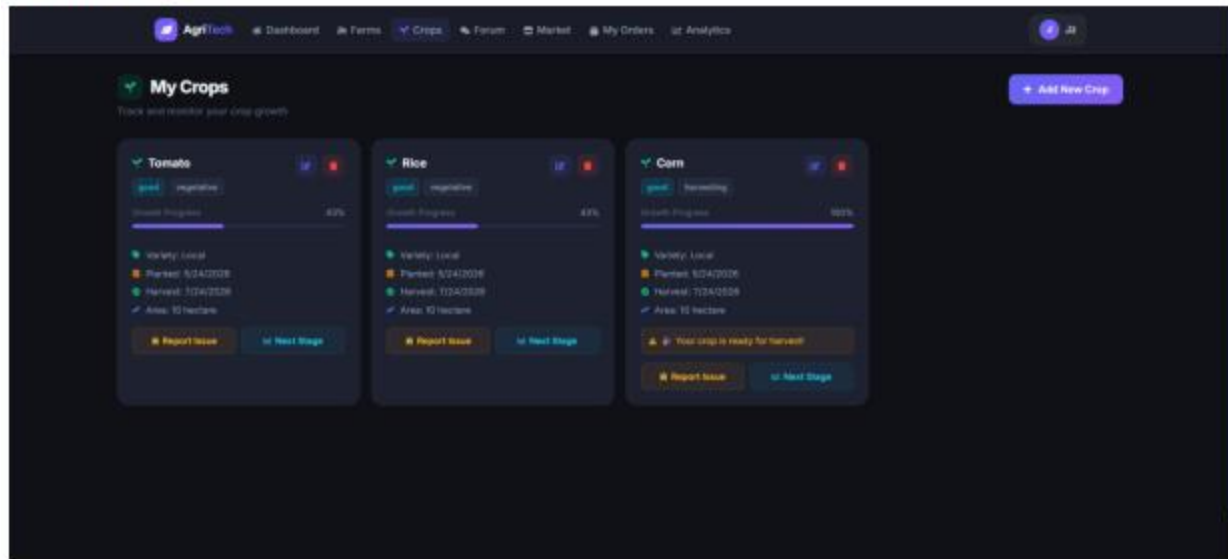
Login Page:



User Home Page:



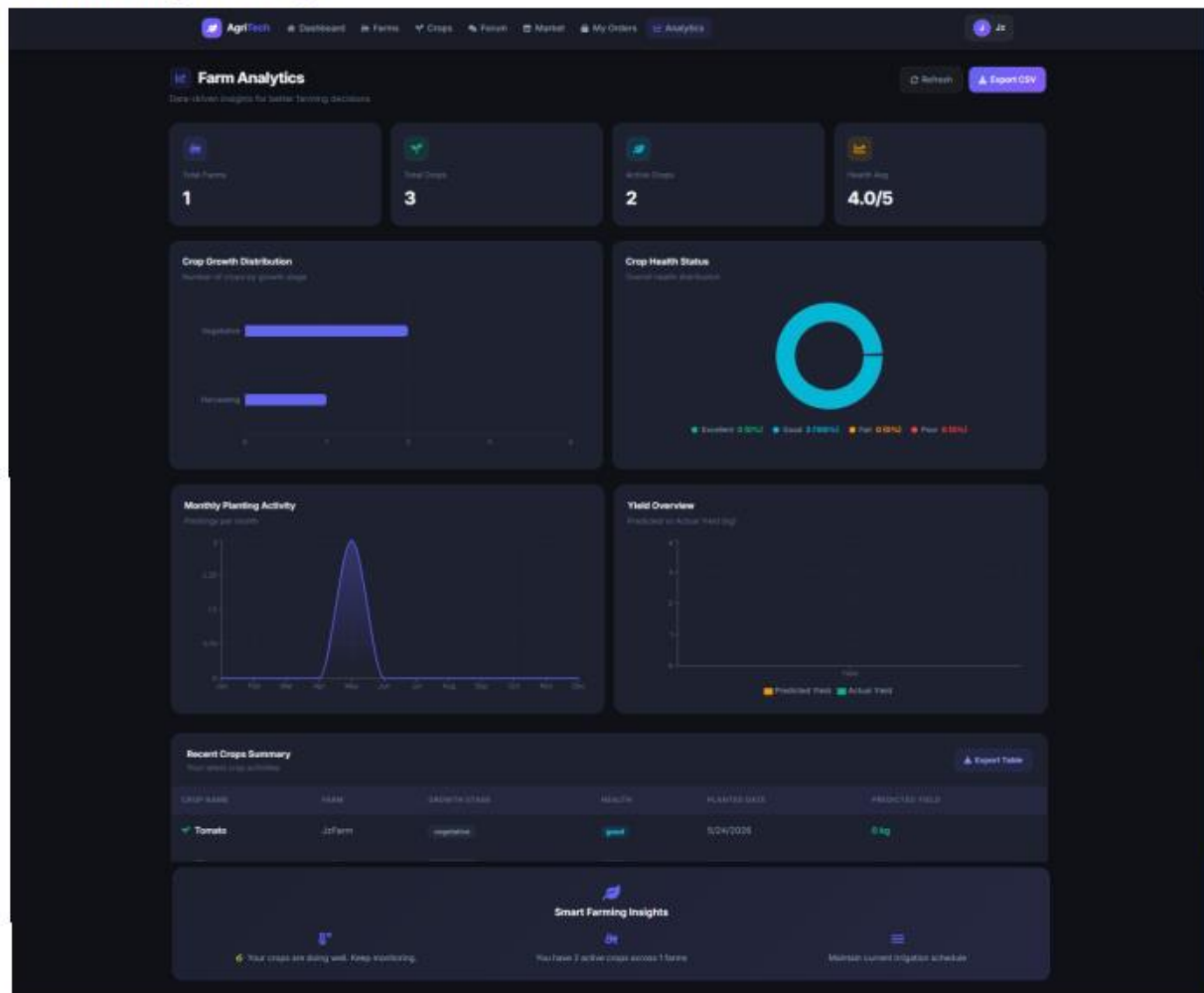
Farms Page:



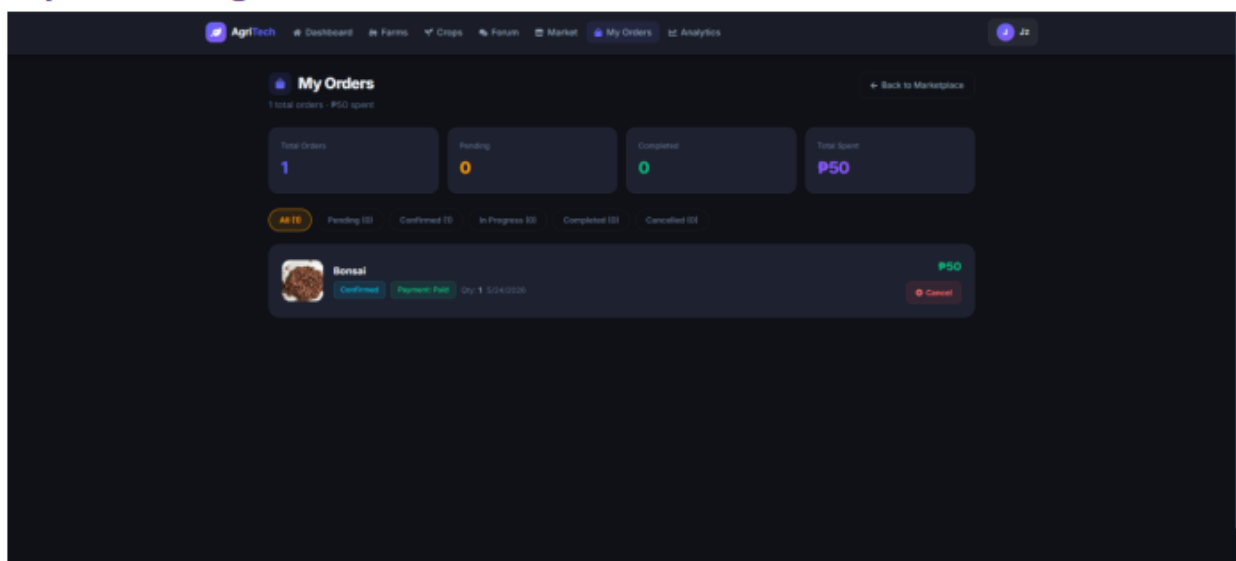
Add Crop:

Add Farm:

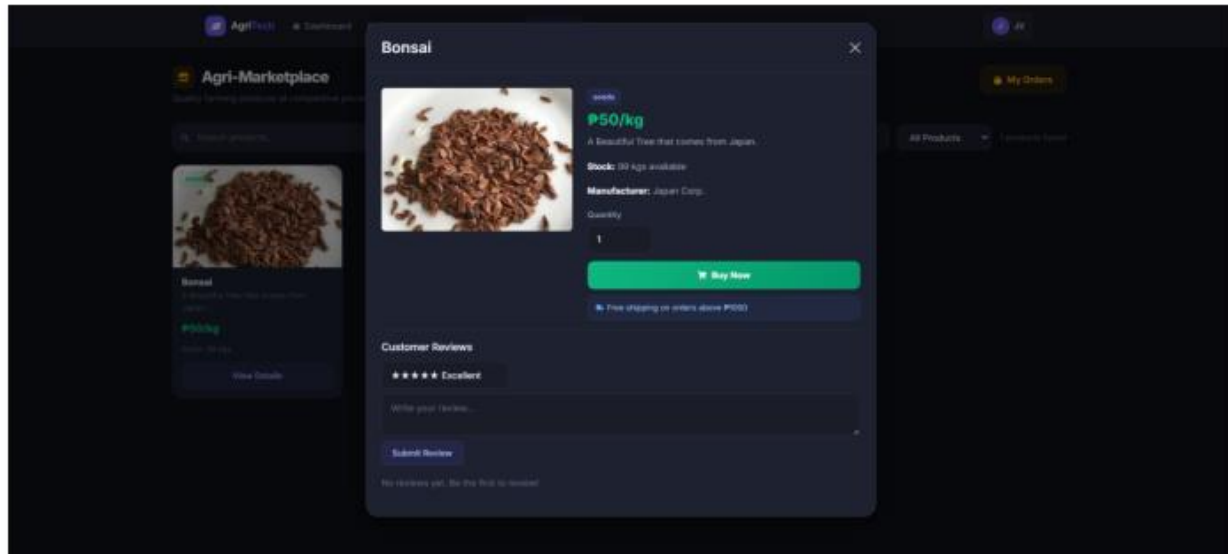
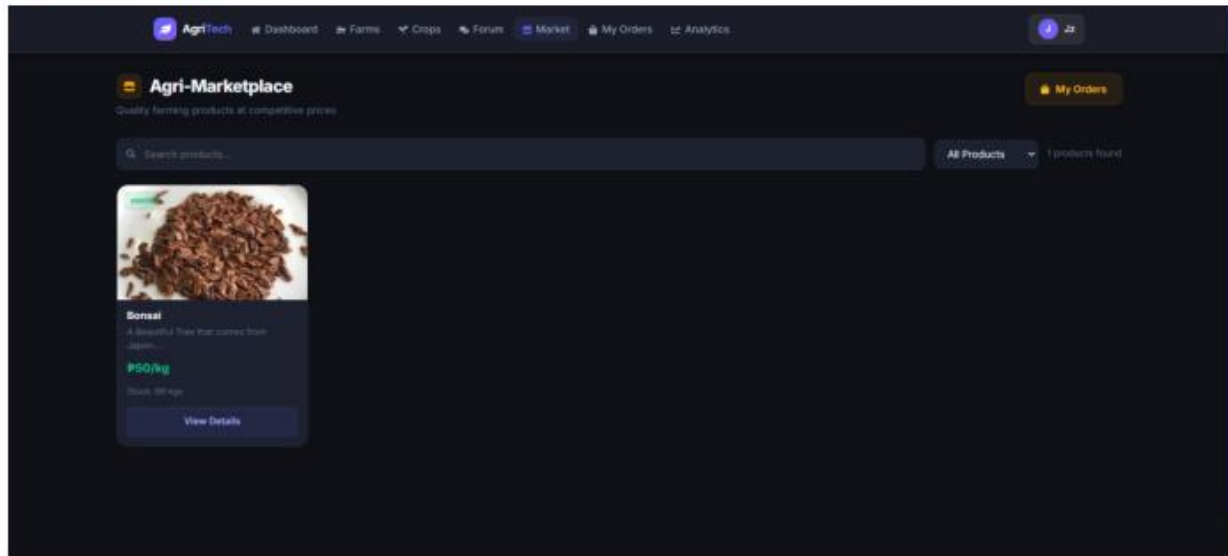
Farm Analytics Page:



My Orders Page:



Market Page:



Admin Products:

Admin Panel AgriTech System

Content & Products AgriTech Admin - Tuesday, May 26, 2026

Content & Products
Manage marketplace inventory and product information

[+ Add Product](#)

Total Products: 1 | Available: 1 | Out of Stock: 0 | Low Stock (1/10)

Search products...

PRODUCT	CATEGORY	PRICE	STOCK	STATUS	RATING	ACTIONS
Banana Jashn Corp	Seeds	\$55/kg	50 kg	Available	★ 5.0	Edit

QUICK STATS

- Users: 9
- Farms: 2
- Crops: 4
- Bookings: 2

[Logout](#)

Admin Users:

Admin Panel AgriTech System

User Management AgriTech Admin - Monday, May 25, 2026

User Management
0 registered users - 0 pending verification

[Create User](#)

Search name or email

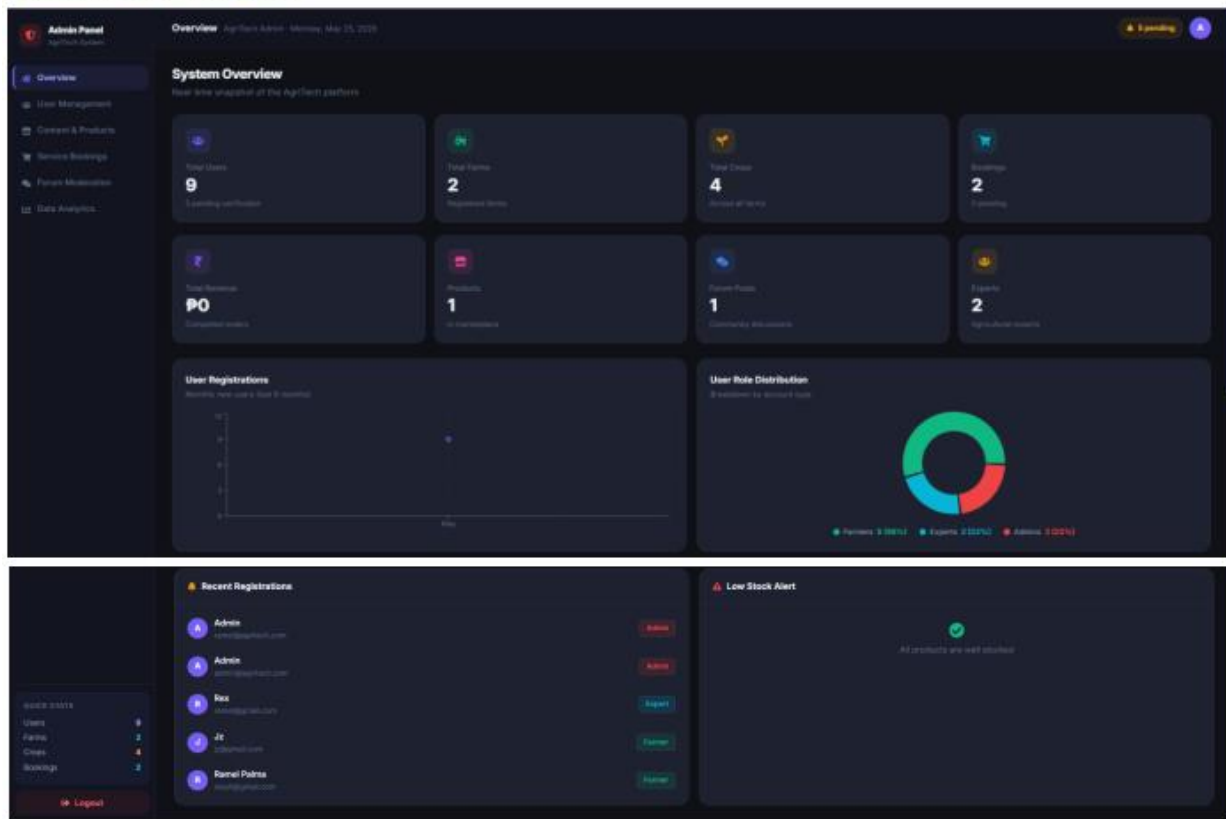
USER	EMAIL	ROLE	STATUS	JOINED	ACTIONS
Admin	ramet@agritech.com	Admin	Verified	5/24/2026	Edit Delete Verify Unverify
Admin	admin@agritech.com	Admin	Verified	5/24/2026	Edit Delete Verify Unverify
Rex	ramet@gmail.com	Expert	Verified	5/24/2026	Edit Delete Verify Unverify
Jz	jz@gmail.com	Farmer	Verified	5/24/2026	Edit Delete Verify Unverify
Ramet Palma	reyyl@gmail.com	Farmer	Verified	5/19/2026	Edit Delete Verify Unverify
jtsading	jtsading@gmail.com	Farmer	Verified	5/19/2026	Edit Delete Verify Unverify
Browser Test	browser@test.com	Farmer	Verified	5/19/2026	Edit Delete Verify Unverify
Ramet Palma	rametpalma980@gmail.com	Farmer	Verified	5/19/2026	Edit Delete Verify Unverify
Rexyl Santiago	reyylsantiago21@gmail.com	Expert	Verified	5/19/2026	Edit Delete Verify Unverify

QUICK STATS

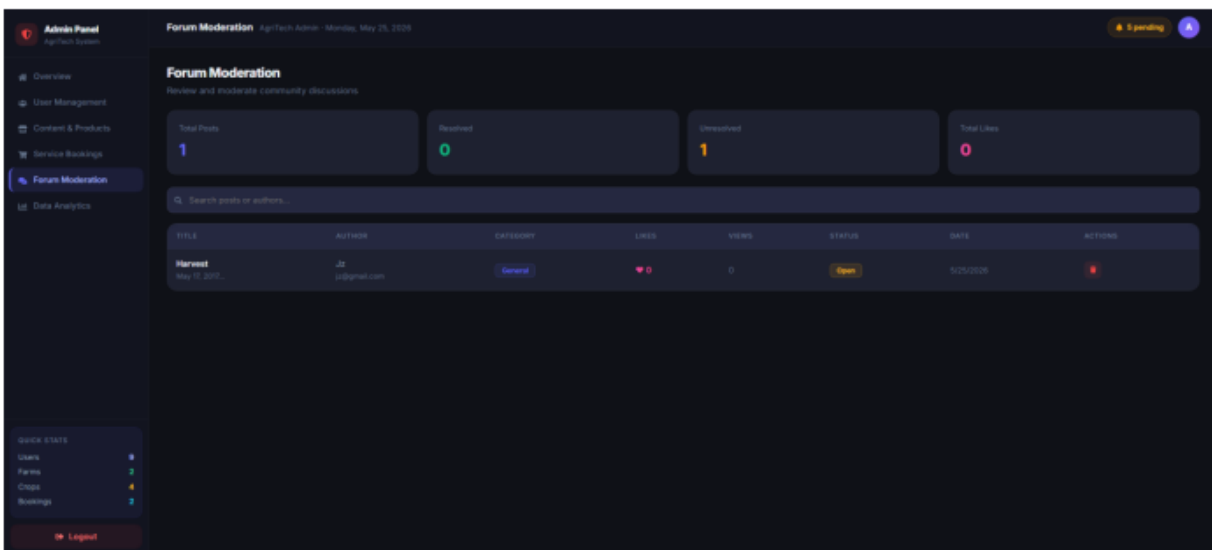
- Users: 9
- Farms: 2
- Crops: 4
- Bookings: 2

[Logout](#)

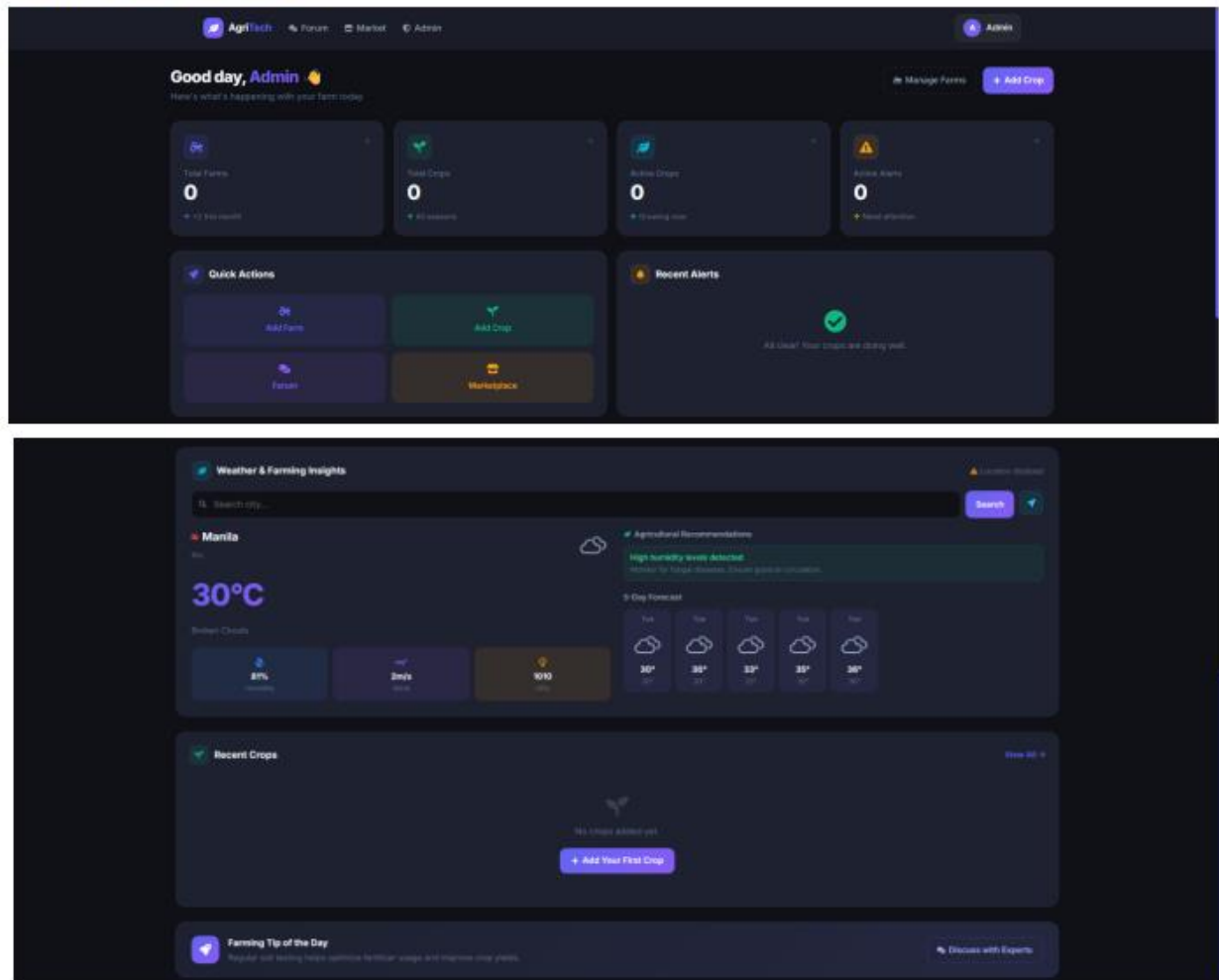
Admin Overview:



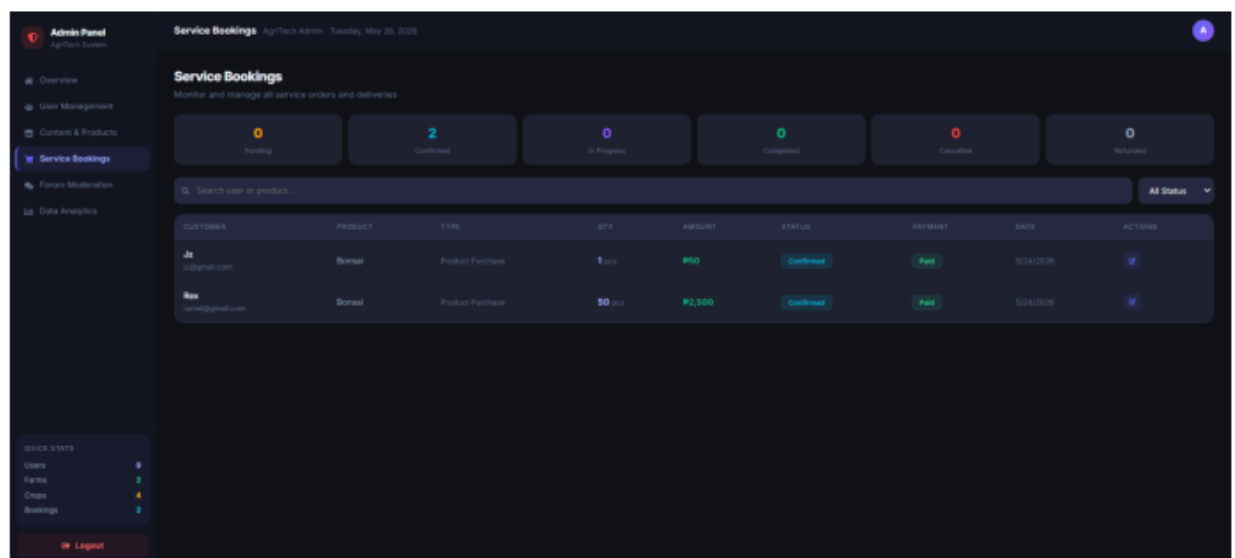
Admin Community Forum:



Admin Dashboard:



Admin Bookings:



To access a live demo, run the application locally using the steps in Section 6 and navigate to <http://localhost:5173>.

12. Known Issues

Issue	Description & Workaround
DNS resolution on Windows	MongoDB Atlas connection may fail on some Windows machines. The server applies a DNS override using Google DNS (8.8.8.8). Ensure MongoDB Atlas Network Access allows your IP (0.0.0.0/0 for development).
Port conflict on 5000	If port 5000 is in use, change PORT= in the .env file to 5001 or any available port.
Weather widget city search	City names must be spelled correctly in English. Abbreviations or local dialect names may not resolve via OpenWeatherMap.
Image upload size limit	Images are limited to 5MB. Uploads larger than this will return an error. Compress images before uploading.
Empty analytics on new accounts	The analytics charts display no data until crops and weather data have been logged. Add crop records to populate the charts.
Forum empty components	Some Forum sub-components (CommentSection, PostDetails) are placeholder files and may require additional implementation.

13. Future Enhancements

The following features and improvements are planned for future versions of the AgriTech Farm Management System:

13.1 Technical Improvements

- Email verification for new farmer accounts using Nodemailer
- Password reset via email OTP flow
- Pagination and search filters on the marketplace and forum pages
- Real-time notifications using Socket.io for booking status updates
- Unit and integration test coverage using Jest and React Testing Library
- Docker containerization for simplified deployment

13.2 Feature Additions

- Crop disease detection using AI image analysis integration
- Government agricultural subsidy information lookup for farmers
- Mobile app version using React Native
- Export farm and crop reports to PDF or Excel format
- Multi-language support (Filipino/English)
- Payment gateway integration (GCash, PayMaya) for the marketplace
- SMS notifications for booking confirmations and weather alerts
- Expert consultation booking with calendar scheduling

13.3 Infrastructure Improvements

- CI/CD pipeline setup using GitHub Actions
- Cloud deployment on platforms such as Render, Railway, or AWS
- CDN integration for uploaded product and profile images
- Rate limiting and request throttling for API security

--- End of Documentation ---